



University
of Glasgow

DayOfWeek DayOfMonth Month 2015
XX.XX am/pm – XX.XX am/pm
(Duration: 1 hour)

DEGREES OF MSc, MSci, MEng, BEng, BSc, MA and MA (Social Sciences)

Big Data: Systems, Programming and Management 4

(Answer all five questions)

This examination paper is worth a total of 60 marks

INSTRUCTIONS TO INVIGILATORS

**Please collect all exam question papers and exam
answer scripts and retain for school to collect.
Candidates must not remove exam question papers.**

1. Describe the read and write protocols, when accessing files in the Hadoop distributed file system (DHFS).

[15]

a. The Write Protocol [8 points – one for each item below]

1. Client tells Name Node (NN) it plans to write (to) a new file
 - By create or open the file for append
2. NN will
 - create a blockID for first new block of the file
 - Decides which DNs will host this blocks' replicas
3. Client receives from NN
 - A lease, something like a lock --- 1 writer many readers
 - The blockID and the Data Node (DN) locations it should go to.
4. Client creates a pipeline, of DNs, based on proximity to client minimizing total network distance to last DN.
5. Client pushes data in packets into a local packet buffer (64KB)
6. When buffer is full, packets are sent to the pipeline
7. Client repeats all above steps for each block of the file
8. Packets are sent asynchronously
 - Next packet is sent before previous is acked

Application can ensure that before sending next packet, the previous one has been received by using the hflush. All packets written before hflush are certain to be visible to readers.

b. The Read Protocol [2 points per item, except last bullet worth 1 point]

1. Client communicates with NN to fetch
 - List of blocks of the file, and
 - DN list with replicas of the file's blocks.
2. Then selects a DN per block and contacts each DN directly asking for blocks
 - DN selection is based on distance: try closer replica first, then if unsuccessful the next, etc.
3. Each DN responds to client with
 - The local file, storing requested block contents
 - The metadata file (including checksum)
- Reading a block from a DN may fail:
 - DN is down or unreachable
 - DN block is "corrupted" i.e., checksum check fails
 - DN may no longer hold a replica of this block !

2. Assume the Name Node in HDFS crashes. What are the resources the NN uses for this purpose and how?

[6]

NN recovery: [2 points describing each resource]

The NN has basically 3 resources at its disposal to use for recovery: the **image**, the **journal**, and the **checkpoint**.

The image is an in-memory resource holding the state of the NN (basically the filename space).

Periodically, to facilitate recovery, the contents of the in-memory image is dumped to stable storage (disk) for durability. The checkpoint is this resource.

In between checkpointing of the image, all new operations mutating the NN state are recorded in a log in stable storage (disk), called the journal. This log is write-ahead, meaning that first operations are saved and then performed on the NN state.

With these resources at hand, upon recovery, the NN re-initializes its state to the last saved checkpoint and then replays the operations in the journal on this image.

HBase logical data model. [all out bullets worth same points – total grade will be the % of coverage of all bullets]

Data model is a sparse, distributed, persistent, multidimensional, sorted map

- Provides a table-like abstraction BUT it is NOT a RDBMS
- No transactions
- No normalization
- No relational API
- Instead, users can define their own extensible, flexible, table-like schema.
- Data is addressable by:
 - Row key (required)
 - Column name
 - timestamp

ie a 3-d map

Each table cell value is just an (uninterpreted) byte array.

Rowkeys are strings.

Ops on single rows are atomic

But no atomicity for ops spanning rows

Rows are lexicographically sorted based on keys

- Columns are grouped into column families. (CFs).
- Each app / user defines its own set of CFs when defining the table.
- Small no. of CFs (e.g. < 10)
- Up to millions of columns within a CF, though
- A column key is named as: family:qualifier
- Access control based on CFs .
- A table is partitioned (split) into tablets
- A tablet is a horizontal partition of a table
- It contains ALL rows in between a *start rowkey* and an *end rowkey*.
Each tablet is assigned to (stored and managed by) a tablet server.
- Each table cell may have multiple versions
- Each associated with a TS
- A TS may reflect real time, or some user-defined
- Cell versions are indexed by timestamp
- Latest versions of each cell are returned by default
- User can also specify
- Explicit no. of desired versions for the system to keep, or
- All versions of a time period (eg last week).

Basic ops:

- **Put**(row_key, column_key, timestamp, value)
- **Get**(row_key) : returns all cells of row
- **Get**(row_key, column_key, timestamp): returns a specific cell
- **Scan**(start_row_key, end_row_key): returns all rows with keys between *start_key* and *end_key*.

In RDBMS lingo, basically selections and projections.

4. What are the main entities and their roles in the HBase system?

[6 points for each of master, and tablet servers, and 3 points for HDFS]

- HBase entities:
 - A single master server
 - Several tablet servers
- HBase depends on a distributed service for
 - Scheduling, resource management, monitoring, and failures
 - Locking services, global data schemas, access control, etc.
- Servers are built upon:
 - (HDFS) and SSTables
 - (HDFS is an open source implementation of GFS)
 - SSTables are “*persistent, ordered, immutable map*” of keys to values, indexed and stored in local file system.

THE MASTER:

- Creates a new tablet / Deletes a tablet / merges two tablets
- Assigns tablets to tablet servers
- Detects new or ‘expired’ tablet servers
- Load balancing among tablet servers
- Schema management (eg table, CF, or column addition/deletions)
- Master is NOT part of client-server data flows !
 - the only way to not have scalability issues in master architectures !

THE TABLET SERVER

- Stores and manages tablets
 - 10s to 1000s of tablets per tablet server
- Serves requests to read and write its own tablets
 - Originally, there is one tablet per table,
 - which is split as table grows (eg > 100-200 MB)
 - Tablets are split as they continue to grow
- Tablet servers can be added / removed dynamically.
Table servers splits tables, while master merges, creates, and deletes them.

Read (get) / Write (put) protocol for a replicated big data system for reads to always see the latest/current version of a data item. [2 points for each protocol and 3 for advantages/disadvantages discussion].

Many read/write protocols are possible. Below 3 major ones are given with pros and cons.

READ (GET) PROTOCOL 1: Read requests are sent to a known primary replica (server).

WRITE (PUT) PROTOCOL 1: Write requests are sent to the primary replica (server).

Primary can be a performance bottleneck and a single point of failure.

READ (GET) PROTOCOL 2: Read requests are sent to any replica (server).

WRITE (PUT) PROTOCOL 2: Write requests are sent to all replicas (servers). If and only if all replicas ack the write successfully, then the write is accepted/acked to the writing application. Else, the write is not accepted by the system. This requires a two-phase process, whereby in the 1st phase replicas are asked to temporarily perform the write, but not commit it. In the 2nd phase, if all replicas have replied ok, then the writer instructs all servers to commit their write locally. Before doing the 2nd phase, the writer must write in stable storage a record with his decision to proceed to commit. This ensures that if he fails in between the 2 phases, upon recovery if a commit record is found in the writer's stable storage, the 2nd phase will be repeated until all replicas have acked their commits.

With these protocols, read requests are made highly available (as any replica can be read), but write availability is penalized (since all replicas must be available). Similarly, reads can be faster (as the nearest copy can be read) while writes take too much time and resources.

READ (GET) PROTOCOL 3 and WRITE (PUT) PROTOCOL 3: This is similar to above, but instead of reading from any and writing to all, a reader must read from R replicas and a write must write to W replicas, where $R+W > N$, and $W+W > N$, where N is the total number of replicas.

This allows a balance between read vs write availability and performance and can tune R and W values per data item, based on expected access patterns.